



# CSCI 112

## Programming with C

### LECTURE: 4

J. L. Pach

# OUTLINE:

- ✂ Statements & expressions
- ✂ Block
- ✂ Accessing Array Elements
- ✂ Logical vs Bitwise Operators
- ✂ Sizeof()
- ✂ const
- ✂ Limits & float
- ✂ Input / Output

# STATEMENTS

*& expressions*

# IMPERATIVE PROGRAMMING

- ✂ Imperative programming is a programming paradigm that uses **statements to change a program's state.**
- ✂ It is like giving the computer instructions step by step on what to do.

Imperative programming

Procedural programming

Object-oriented

# PROCEDURAL PROGRAMMING

- ✂ **Procedural programming** is a type of imperative programming that breaks down a program into smaller, independent procedures (functions).
- ✂ This makes the program more organized and easier to maintain.

Imperative programming

Procedural programming

Object-oriented

# OBJECT-ORIENTED PROGRAMMING

- ✂ **Object-oriented programming** further extends the concepts of procedural programming by organizing code into objects that encapsulate data and behavior.

Imperative programming

Procedural programming

Object-oriented

# STATEMENT & EXPRESSION

- ✂ In computer science, a **statement** is a complete instruction that can modify the state of a program, such as assigning a value to a variable, calling a function, or jumping to a different part of the code. Statements do not produce a value on their own.
- ✂ An **expression**, on the other hand, is a combination of variables, constants, functions, and operators that evaluates to a single value (**The absence of a returned value from a function is also a result**). Expressions can be used within statements to provide values for operations or to control the flow of the program. However, expressions themselves do not directly modify the state of the program.

# NULL / EMPTY STATEMENT;

```
int main()
{
    ;                /* Empty statement */
    printf("Hello world!\n");    /* Second statement */
    printf("Hello"); printf(" world!\n");    /* 3rd & 4th statement in one line */
}
```



- ✚ A single semicolon `;` in C language is a statement called a `null` statement or empty statement.
- ✚ A semicolons `;` are used to terminate statements

# CONCLUSIONS

- ✂ *In C, a code line is not the same as a statement*
- ✂ *An entire program can be written in a single line*
- ✂ *As the compiler separates statements with semicolons ; and not newlines*
- ✂ *Therefore, a single line of text (with a newline character) can contain multiple statements.*

**BLOCK**

# BLOCK & COMPLEX(COMPOUND) STATEMENTS

- ✂ **Block** can be:
  - ✂ substitute for simple statement,
  - ✂ defined within a block (Nested blocks),
  - ✂ empty `{}`.
- ✂ Variables can be declared inside
- ✂ Variables declared inside a block with the same name hide access to variables outside that block
- ✂ No semicolon at end of block
- ✂ After a code block ends, all variables declared within that block cease to exist and are no longer accessible in the rest of the program.

# BLOCK & COMPLEX(COMPOUND) STATEMENTS - EXAMPLE

```
int main()
{
    char* text = "Hello world\n";
    printf(text);           // Hello world
    {
        char* text = "Bye world\n";
        printf(text);       // Bye world
        {
            char* text = "Aloha world\n";
            printf(text);     // Aloha world
        }
    }
    printf(text);           // Hello world
}
```



## SUMMARY

- ✖ A **statement** can modify the state of a program, such as assigning a value to a variable, calling a function, or jumping to a different part of the code.
- ✖ Statements do not produce a value on their own.
- ✖ An **expression** is a combination of variables, constants, functions, and operators that evaluates to a single value.
- ✖ Expressions themselves do not directly modify the state of the program.

# EXAMPLE

```
int main()
{
    int x, y, _;
    _ = x = y = 5;

    _ = x = y = 5;
    _ = x = (5);
    _ = x = 5;
    _ = (5);
    _ = 5;
    (5)
}
```



# CONSEQUENCES

- ✖ In the C language, **every** assignment after assigning a value to a variable (pointer, label), since it is also an expression, will generate a value again, which will be ignored by the compiler at the end.
- ✖ We can use the underscore `_` discard in C, but you need to declare the variable beforehand. It's a great mechanism that improves the readability of the programmer's intent.

# ACCESSING

*Array Elements*

# ACCESSING ARRAY ELEMENTS 1/2

```
#include <stdio.h>
int main()
{
    int tab[3];    /* Declare an array of 3 integers */
    /* Store values into the array */
    tab[0] = 5;    /* put 5 into the first element */
    tab[1] = 10;   /* put 10 into the second element */
    tab[2] = 15;   /* put 15 into the third element */
    /* Retrieve values from the array */
    int x = tab[0]; /* read the first element into variable x */
    int y = tab[1]; /* read the second element into variable y */
    /* Display the values */
    printf("x = %d\n", tab[0]); // prints 5
    printf("y = %d\n", tab[1]); // prints 10
    return 0;
}
```



## ACCESSING ARRAY ELEMENTS 2/2

- ✚ `tab[index]` is used both to read and write values.
- ✚ Array indices in C start from 0.
- ✚ Make sure the index is within the declared size.
- ✚ Each element in the array is stored contiguously in memory.

```
struct Point { int x; int y; }; int main()
{struct Point point = {1,2}, *ppoint = &point; int arr[] = {1,2}; int x = 5, y =-6; int * z; float f = 3.0f; /*code*/}
```

| Priority / Operator |                      | Description                                                 | Associativity        | Example                 | Result       |
|---------------------|----------------------|-------------------------------------------------------------|----------------------|-------------------------|--------------|
| 1                   | <b>(), []</b>        | <b>Parentheses; Array subscript</b>                         | <b>Left-to-right</b> | <b>arr[0] * (x + y)</b> | <b>1</b>     |
|                     | .                    | Structure and union member access                           |                      | point.x                 | 1            |
|                     | ->                   | Structure and union member access through pointer           |                      | ppoint->x               | 1            |
| 2                   | ++, --               | Prefix & postfix increment and decrement                    | Right-to-left        | ++x; x--; x;            | 6, 6, 5      |
|                     | +, -, !, ~           | (Unary) plus and minus; Logical NOT and bitwise NOT         |                      | y =-y; y =+y; !x; ~x    | 6, -6, 0, -6 |
|                     | *, &, &&             | Indirection (dereference); Address-of; Address-of labels    |                      | z = &x; *z;             | 6422276; 5   |
|                     | (type), sizeof       | Cast, Size-of                                               |                      | (int)3.0f; sizeof(x);   | 3, 4         |
| 3                   | *, /, %              | Multiplication, division, and remainder                     | Left-to-right        | 6/2 % 2                 | 1            |
| 4                   | +, -                 | Addition and subtraction                                    |                      | 1 + 2; 3 - 1            | 3, 2         |
| 5                   | <<, >>               | Bitwise left shift and right shift                          |                      | 4 << 1; 4 >> 2          | 8, 1         |
| 6                   | <, <=, >, >=         | For relational operators <, > and ≤, ≥ respectively         |                      | x<y; x<=y; x>y; x>=y    | 1, 1, 0, 0   |
| 7                   | ==, !=               | For relational = and ≠ respectively                         |                      | x == y, x != 1          | 1, 0         |
| 8                   | &                    | Bitwise AND                                                 |                      | 7 & 3                   | 3            |
| 9                   | ^                    | Bitwise XOR (exclusive or)                                  |                      | 255 ^ 0                 | 255          |
| 10                  |                      | Bitwise OR (inclusive or)                                   |                      | 7   3                   | 7            |
| 11                  | &&                   | Logical AND                                                 |                      | 1 && 0                  | 0            |
| 12                  |                      | Logical OR                                                  |                      | 1    0                  | 1            |
| 13                  | ?:                   | Ternary conditional                                         | Right-to-left        | x = (x > y) ? y : x;    | -6           |
| 14                  | =                    | Simple assignment                                           |                      | x = y;                  | -6           |
|                     | +=, -=, *=, /=, %=   | Assignment by sum, difference, product, quotient, remainder |                      | x+=1; x-=1; //etc.      | 6, 5         |
|                     | <<=, >>=, &=, ^=,  = | Assignment by bitwise left shift, right shift, AND, XOR, OR |                      | 3<<=1, 8>>=2 //etc.     | 6, 2         |
| 15                  | ,                    | Comma                                                       | Left-to-right        | x = 3, y = 1;           | 3            |

# LOGICAL

vs Bitwise Operators

# LOGICAL VS BITWISE OPERATORS

In C, Boolean logic can be used in two ways:

## Bitwise operations on variables:

- ✂ the result is a numeric value obtained by applying the corresponding logic gate to each bit of the variables involved. The exception is the bitwise NOT `~`, which inverts all bits.

## Evaluating the logical value of an expression:

- ✂ remember that any non-zero value (including negative numbers) is treated as `true` (1), and zero (or NULL) is `false` (0). Essentially, the numeric value of each variable is first cast to `0` or `1`, and then the logical expression is evaluated.

# BITWISE OPERATORS

```
unsigned int x = 6;          /* 00000000 00000000 00000000 0000 0110 (binary) */
unsigned int y = 3;          /* 00000000 00000000 00000000 0000 0011 (binary) */

/* AND & */
printf("%u\n", x & y);       /* (00000000 00000000 00000000 0000 0010) */      //2
/* OR | */
printf("%u\n", x | y);       /* (00000000 00000000 00000000 0000 0111) */      //7
/* NOT ~ (flips all bits) */
printf("%u\n", ~x);          /* (11111111 11111111 11111111 1111 1001) */      //4 294 967 289
/* XOR ^ */
printf("%u\n", x ^ y);       /* (00000000 00000000 00000000 0000 0101) */      //5
```



# LOGICAL OPERATORS



```
unsigned int x = 6;          /* 00000000 00000000 00000000 0000 0110 (binary) */
unsigned int y = 3;          /* 00000000 00000000 00000000 0000 0011 (binary) */

/* AND && */
printf("%d\n", x && y);      /* (00000000 00000000 00000000 0000 0001) */    // 1 (true)
/* OR || */
printf("%d\n", x || y);     /* (00000000 00000000 00000000 0000 0001) */    // 1 (true)
/* NOT ! */
printf("%d\n", !x);         /* (00000000 00000000 00000000 0000 0000) */    // 0 (false)
/* XOR doesn't exist but = (a || b) && !(a && b); */
printf("%d\n", (x || y) && !(x && y)); /* (00000000 00000000 00000000 0000 0000) */    // 0 (false)

printf("%d\n", -1 && -2);    /* (00000000 00000000 00000000 0000 0001) */    // 1 (true)
```

**sizeof()**

# SUMMARY OF MEMORY SIZE OF DATA TYPES

| Type                                                                                      | Memory size in bytes / bits |
|-------------------------------------------------------------------------------------------|-----------------------------|
| char                                                                                      | 01 Bytes / 08 bits          |
| bool *                                                                                    | 01 Bytes / 08 bits          |
| short int                                                                                 | 02 Bytes / 16 bits          |
| long int                                                                                  | 04 Bytes / 32 bits          |
| float                                                                                     | 04 Bytes / 32 bits          |
| double                                                                                    | 08 Bytes / 64 bits          |
| long double                                                                               | 12 Bytes / 96 bits          |
| pointer ( char *, short *, long *, int *, float *, double *, long double *, void *, ... ) | 04 Bytes / 32 bits          |
| user defined ( struct , union , etc. )                                                    | complex                     |

\*from C99

## OPERATOR sizeof()

The sizeof operator returns the size of a **type**(requires parentheses) or **variable** in bytes.

- ✂ It is a **unary operator**, just like a cast, and has the same precedence level.
- ✂ It is easy to mistake sizeof for a function, but it is not — it is evaluated at compile time.

```
long long x, short y, float z;
printf("%d\n", sizeof(x));      // 8
printf("%d\n", sizeof x );      // 8
printf("%d\n", sizeof(long long)); // 8
printf("%d\n", sizeof(y));      // 2
printf("%d\n", sizeof y );      // 2
printf("%d\n", sizeof(short) ); // 2
printf("%d\n", sizeof(z));      // 4
printf("%d\n", sizeof z );      // 4
printf("%d\n", sizeof(float) ); // 4
```



CONST

# NAMED CONSTANTS IN C

When you see names written in **ALL CAPS** in C code, they represent **constants** — values that do not change during program execution.

## The `const` Keyword

You can create a constant by adding `const` before the type:

```
const int MAX_STUDENTS = 30;  
const double PI = 3.14159;
```



- ✂ `const` tells the compiler: this value must never be changed.
- ✂ If you try to modify it, the compiler will report an error.

# NAMING CONVENTION

Constants are written in `ALL_CAPS_WITH_UNDERSCORES`.

```
const int MAX_SIZE  = 100;  
const int MIN_AGE   = 18;  
const double GRAVITY = 9.81;
```



This is a widely used convention in C — it makes constants easy to spot and distinguish from regular variables at a glance.

## Why Does This Matter?

When you see names like `INT_MAX` or `FLT_MAX` in C, they follow exactly the same convention — they are constants that represent the largest value an int or float can hold.

# LIMITS & FLOAT

## LIMITS.H & FLOAT.H

- ✖ To access constants defining the minimum and maximum values of integer types in C,
  - ✖ include the header `<limits.h>`,
  - ✖ For floating-point types, include `float.h`.
- ✖ Important: `FLT_MIN` (and other floating-point *min* constants) does not represent the most negative value. It is the smallest positive non-zero value that can be represented.

If you want the actual minimum (most negative) value, use `-FLT_MAX`.

# LIMITS.H

```
printf("CHAR_MIN      = %d\n", CHAR_MIN);      /* char */
printf("CHAR_MAX      = %d\n", CHAR_MAX);
printf("UCHAR_MAX     = %u\n", UCHAR_MAX);

printf("SHRT_MIN      = %d\n", SHRT_MIN);      /* short */
printf("SHRT_MAX      = %d\n", SHRT_MAX);
printf("USHRT_MAX     = %u\n", USHRT_MAX);
```



Result:

```
---
CHAR_MIN      = -128
CHAR_MAX      = 127
UCHAR_MAX     = 255
SHRT_MIN      = -32768
SHRT_MAX      = 32767
USHRT_MAX     = 65535
```

# FLOAT.H

```
printf("INT_MIN      = %d\n", INT_MIN);      /* int */
printf("INT_MAX      = %d\n", INT_MAX);
printf("UINT_MAX     = %u\n", UINT_MAX);

printf("FLT_SMALLEST = %e\n", FLT_MIN);      /* float */
printf("FLT_MIN      = %f\n", -FLT_MAX);
printf("FLT_MAX      = %f\n", FLT_MAX);
printf("FLT_MAX      = %e\n", FLT_MAX);
```



```
INT_MIN      = -2147483648
INT_MAX      = 2147483647
UINT_MAX     = 4294967295
FLT_SMALLEST = 1.175494e-038
FLT_MIN      = -340282346638528859811704183484516925440.000000
FLT_MAX      = 340282346638528859811704183484516925440.000000
FLT_MAX      = 3.402823e+038
```

INPUT / OUTPUT

# INPUT / OUTPUT

Input and output functions in C are not available by default — you need to include the `<stdio.h>` library. The most basic functions include:

- ✂ `getchar()` — reads a single character from the keyboard,
- ✂ `putchar()` — displays a single ASCII character on the screen,
- ✂ `printf()` — prints a formatted string to the screen,
- ✂ `scanf()` — reads a formatted string from the keyboard.

# INPUT / OUTPUT

In C, it **is not** easy to explain how these functions work internally without more advanced knowledge. Therefore, we will start by using only the output functions:

✂ `putchar()` and `printf()` .

To better understand how input works, we will first use the **getche()** function from the Windows-only `conio.h` library. After that, we will move on to `getchar()` and finally to `scanf()` .

Getche() - (get[ ]char[acter with echo])

# FUNDAMENTAL FUNCTIONS FOR INPUT AND OUTPUT

## ✂ Data Output for Screen:

- ✂ `putchar` \* - (`put[ ]char[acter]`): Displays a single character on the screen.
- ✂ `printf` - (`print[ ]f[ormatted]`): Displays a formatted string of characters.

## ✂ Data Input from Keyboard:

- ✂ `getchar` \* - (`get[ ]char[acter]`): Retrieves a single character from the keyboard.
- ✂ `scanf` - (`scan[ ]f[ormatted]`): Reads a formatted string of characters from the keyboard.

✂ C doesn't handle input/output on its own. You need a library called `<stdio.h>`.

✂ \*You can use `putchar` and `getchar` like normal functions, but they are not standard C functions but rather preprocessor macros.

## PUTCHAR() & GETCHE() WITHOUT \N

```
int main()
{
    char character;
    character = getche(); /* Read a single key from the keyboard and immediately display it (echo) */
    putchar(character); /* Display the value of 'character' using putchar (prints the same key again) */

    character = getche(); /* Read another single key (char is enough, automatic casting occurs) */
    putchar(character); /* Display the second key pressed */
} // Result: aaBB
```



```
int putchar(int);
```

- ✂ `putchar(c)` puts the character `c` on the standard output. It returns the character printed or EOF on error (-1).

```
int getche(void);
```

- ✂ returns the next character from standard input. It returns EOF (end of file) on error.

## PUTCHAR() & GETCHE() WITH \N

```
int main()
{
    char character;
    character = getche();    /* Read a single key from the keyboard and immediately display it (echo) */
    putchar('\n');          /* Display the CRLF using putchar to move to the next line for clarity */
    putchar(character);     /* Display the value of 'character' using putchar (prints the same key again) */
    putchar('\n');          /* Display the CRLF using putchar to move to the next line for clarity */

    character = getche();    /* Read another single key (char is enough, automatic casting occurs) */
    putchar('\n');          /* Display the CRLF using putchar to move to the next line for clarity */
    putchar(character);     /* Display the second key pressed */
    putchar('\n');          /* Display the CRLF using putchar to move to the next line for clarity */
}
```



a

a

B

B

## CHAR ISSUE 1/2

- ✖ By default, `char` in C is `signed`, which means its range is -128 to 127 (two's complement) instead of 0–255.
- ✖ When you write: `char var = 255;` or something greater than 255+x;
- ✖ you need to know that 255 is by default an int (4 bytes).

## CHAR ISSUE 2/2

- ✖ An automatic cast occurs here: only the lowest 8 bits are kept, and the most significant bit (**MSB**), which determines the sign, is overwritten. As a result, `printf()` will show `-1` because that is how these 8 bits are interpreted in signed form.
- ✖ This is an example of overflow and truncation, causing loss of information. This is also why `getche()` / `getchar()` returns an `int`: it can hold all possible `char` values plus the special `EOF` value `-1`. Using `int` ensures we can reliably distinguish between a real `char` with all bits set and an intentional `EOF` signal.

## BASICS OF INT PRINTF (CHAR FORMAT[], ARG1, ARG2 ,...);

- ✂ The format requires a string enclosed in double quotes ( "<string>" ).
- ✂ If we want to display the contents of our variables, such as `int` types, we must use the `%` symbol followed by the type. This allows displaying a value from memory interpreted as the given type and after we must provide the name of the variable that will be read. Each instance of `%<type>` will allow us to display the contents of one variable.

```
int main()
{
    int x = 5;
    int y = 7;
    printf ("%s\n", "Hello world\n");           // Hello world
    printf ("%d\n", x);                          // 5
    printf ("Value of x = %d, and value of y = %d\n", x, y); // Value of x = 5, and value of y = 7
    printf ("%d %d\n", x, y, x, x);              // 5 7
}
```



THANK

*You*